Python 练习册 · 进阶篇

学完《Python 入门》教科书与基础篇后开练。4 个必做场景（+1 个选做）= 五份工单，答案全部集中在书末。

> **做完之后：**本册所有阶段做完，请打开《**17-关卡总表**》对照本册部分逐条打勾——**打勾 = 真正会了**（不是"看过"）；卡住的条目回到对应阶段重做。 打勾全部完成，就可以进入下一册。

本册关卡地图（先读这一页，1 分钟）

项目	内容
标签	**课程 + 字典** —— 课程（作品工单）
本册关卡	3-11 ~ 3-17
用时	必做 25 小时
学完解锁	三个能上传 GitHub 的脚本 + 王牌项目第一版（本机巡检 v0）

> **场景 5（paramiko）留到以后再做；参考答案约 11,000 字，做完之前不许翻。**
>
> 关卡编号、时长、完成判定都写在正文每章/每节的「本关三要素」里；全书进度在《17-关卡总表》。

开始之前 · 先弄懂这是什么（2 分钟）

① **是什么**：这是《基础篇》之后的 Python 进阶练习册 —— 不再是"一题一个知识点"，而是 **真实场景的工单**（必做 4 个：批量重命名 / 天气查询 / 本机巡检 / 日志分析；选做 1 个：远程巡检）：只给背景和目标，怎么拆、怎么写、怎么验证全靠你自己。生活类比：像**实习第一天接到的任务单**——领导只提需求，技术方案你来定。

② **干什么用**：练"**拆需求 → 写代码 → 验证**"的完整流程；产出 3 个能上传 GitHub 的脚本 + 王牌项目第一版（本机巡检 v0）——都是能拿得出手的成果。

③ **为什么要学（+ 实战里怎么用）**：工作里常问"你写过什么"，能答出"我自己写了个巡检脚本，查 CPU / 磁盘、超阈值报警、挂进 crontab"就是硬货；AI 运维的日常活，就是这一册套路的复杂版。

④ **长什么样 / 怎么用**：一个场景走完是这样：读背景 → **先写方案**（想清楚再动手）→ 写代码 → 用测试文件**验证**（如先跑 `--dry-run`）→ 记进学习日志。规矩照书里：**先自己想 3 分钟、真跑一遍、卡住 20 分钟再查。**位置：05 册之后、07 Docker 之前（这几个脚本正是后面容器化的素材）。

使用说明

定位：进阶篇不再是"一题一个知识点"，而是**五个真实场景的工单**——每个场景只给你背景和目标，怎么拆解、怎么写代码、怎么验证，全靠你自己。这就是以后工作的样子：领导只提需求，技术方案你来定。

场景地图（必做场景估时合计约 11.5 小时；本册关卡口径必做 25 小时，见《17-关卡总表》）：

场景	名称	时长	产出（脚本文件）
★ 必读	使用说明（怎么用本册）	5 分钟	—
★ 必读	示范题（先读，锁定质量标准）	约 25 分钟	—
★ 必做	批量文件重命名工具	约 2.5h	脚本 1
★ 必做	天气查询器	约 2.5h	脚本 2
★ 必做	本机巡检工具雏形	约 3h	王牌项目 server-health-check 版本 0
★ 必做	Nginx 日志分析器	约 3h	脚本 3（主线用附录 B 样例日志；Docker 真实日志版为外部扩展）
☆ 选做	paramiko 远程巡检雏形（以后再做；需自有 SSH Ubuntu 环境，无条件可跳过）	约 2h	server-health-check 版本 2 雏形

答题要求：① 先在纸上/脑子里拆解任务，再敲代码；② 每道题必须自己跑通看到输出；③ 对答案时先看"先想什么"和"为什么这么写"，最后看代码；④ 每个场景的最终脚本，都要保存成脚本文件（对照上面"场景地图"的"产出"列：脚本 1 / 脚本 2 / 巡检 v0 / 脚本 3 都是产出，场景 5 选做）；⑤ 每天把敲过的命令、报错与心得记进学习日志。

安全红线：

① 场景 1 涉及**真实重命名文件**：只在专门的测试目录里练，先建测试目录并放"假文件"，正式跑之前必须先 `--dry-run` 试运行；② 场景 2 联网：注意 `timeout` 超时设置，别让程序无限卡住；③ 场景 5（选做、以后再做）涉及 SSH：只连自己的虚拟机，**绝不连接生产服务器**。

依赖准备（要装的第三方库）：必做的场景 2 需要 `requests`（现在就装）；场景 5（选做、以后再做）才需要 `paramiko`，到时再装。装法走虚拟环境四步：`sudo apt install -y python3-venv python3-pip` → `python3 -m venv .venv` → `source .venv/bin/activate` → `pip install requests`（做场景 5 时再加装 `paramiko`）。完整说明与常见报错见《Python 入门》**附录 E（venv 与 pip）**。

实验闭环：涉及重命名、联网、日志、系统命令或 SSH 的场景，先记录原状态；执行前核对路径、目标、参数和远程主机；执行后核对输出、退出码和新增记录；结束后恢复文件、日志、进程和连接。异常时保留原始证据。

参考答案规则：全部答案集中在书末，**题目页不出现任何提示**（示范题除外——它是"标准演示"，不是题目）。每题四步式：先想什么 → 为什么这么写 → 完整代码 → 怎么验证。

示范题（先读这一题，锁定质量标准）

体会四步式答案的写法，后面的题都按这个标准做。（说明：本示范题为独立场景，不与任何正题重叠——展示"四步式答案"的写法标准）

示范题（独立示范，不占任何正题）

本示范用 `whoami`（Windows/Ubuntu/Termux 都有，无需电脑环境——平板也能跑），只演示 `subprocess` 的**最小完整模板**。它不提供其他场景的设计思路或答案；后续任务需要根据各自的题目目标独立完成。

题目：用 `subprocess` 执行 `whoami`，把输出拿到 Python 里打印成 `"当前用户是：xxx"`。

① 先想什么：任务三步——执行命令（`subprocess.run` 传命令列表）、拿输出（返回对象里的 `.stdout`）、处理输出（strip 去掉末尾换行）。

② 为什么这么写：capture_output=**True** 把命令输出抓回来（第 11 章）；text=**True** 让输出是字符串而不是字节串；strip() 去掉输出末尾的换行符——不 strip 的话打印结果会多一个空行。

③ 完整代码：

```
import subprocess

r = subprocess.run(["whoami"], capture_output=True, text=True)
print("当前用户是：", r.stdout.strip())
```

④ 怎么验证做对了：输出"当前用户是：你的真实用户名"，与终端直接敲 `whoami` 的结果一致；把命令换成 `["ls", "-l"]` 同样能拿到输出——说明模板是通用的。

场景 1｜批量文件重命名工具

本关三要素（编号 ｜ 时长 ｜ 完成判定）
3-11 • 场景 1（上）：批量重命名——先想清楚再动手 ★ ｜ 休息日 3 小时 ｜ 工作日 15 分
□ 能写出改名计划（哪些文件、改成什么）
回滚方式：先跑 --dry-run，保留原名映射
3-12 • 场景 1（下）：实现 + 冲突处理 + 验证 ★ ｜ 休息日 4 小时 ｜ 工作日 15 分
□ 测试目录里的文件全部改名成功、无覆盖（附录 A 给的 10 个示例文件即可；想加量就用循环生成——判定看的是"一个不落、一个不覆盖"）
回滚方式：同上
工作日 15 分钟版：只做第 1 步 + 背 1 张卡（卡片见各阶段的工作日卡）。

📋 **背景工单**：你在运维组实习，同事甩来一个文件夹，里面 100 个文件命名混乱（`照片(1).jpg`、`IMG_2024...` 各种格式）。领导要一份工具：以后任何乱命名文件夹都能一键规范化。

🎯 **只给目标**（最终交付一个 `rename_tool.py`）：

目标	要求
A	列出目录所有文件，只处理指定扩展名的文件（扩展名可用 `endswith`（判断以...结尾））
B	按规则生成新名字：统一前缀 + 三位序号（如 `photo_001.jpg`）
C	批量重命名（用 `os.rename`（重命名文件）），且**不得覆盖已有文件**（重名时要有应对）
D	支持 `--dry-run`（试运行）模式：先打印将要执行的重命名清单，确认后才真正执行
E	程序通过 `argparse` 接收目录路径参数

⚠ 安全提示：重命名不可逆。① 只在专门测试目录里练（先建 `~/lab/renametest` 放一堆"假文件"）；② 正式执行前必须先 `--dry-run` 看清单；③ 测试目录建议先手动备份一份。

✎ 测试文件生成命令见附录 A（10 个乱命名文件，一行一个）。产出：脚本 1，保存为脚本文件。

场景 2｜天气查询器

本关三要素（编号 ｜ 时长 ｜ 完成判定）
3-13 • 场景 2：天气查询器（API + 参数 + 异常） ★ ｜ 休息日 4 小时 ｜ 工作日 20 分
□ 换城市能查、断网不炸、结果能存文件
工作日 15 分钟版：只做第 1 步 + 背 1 张卡（卡片见各阶段的工作日卡）。

📋 **背景工单**：办公室要做个命令行小工具，输入城市名就能看当前天气，给巡检同事出门用。

接口信息（题面给出，标注含义）：

项目	说明
接口地址	`https://api.open-meteo.com/v1/forecast`（Open-Meteo 免费天气接口，无需注册）
latitude / longitude	纬度 / 经度（参数名，直接拼进 URL，如 `?latitude=30.25&longitude=120.17`）
current_weather=true	请求当前天气（固定写法）

项目	说明
返回 JSON 结构	current_weather 字段里有 temperature（温度，单位°C）、windspeed（风速）、weathercode（天气代码数字）
weathercode 对照	0=晴、1~2=多云、3=阴、45=雾、51~67 与 80~82=有雨、71~77=有雪、≥95=雷暴
城市经纬度表	北京 (39.90,116.40)、上海 (31.23,121.47)、杭州 (30.25,120.17)、广州 (23.13,113.26)、成都(30.57,104.06)

🎯 **只给目标**（最终交付一个 weather.py）：

目标	要求
A	用 requests 调用上面的天气接口（城市经纬度从上表查）
B	用 json 解析返回数据，提取温度和天气状况（weathercode 转成中文）
C	城市参数通过 argparse 传入，输出一行友好报告（含单位 °C）
D	异常处理全覆盖：网络不通、接口返回错误都要有对应提示，不能裸奔报错
E	结果写入 weather_report.txt（追加一行：城市 温度 天气）

⚠ 安全提示：本题联网。① 请求要设 timeout（超时上限，如 10 秒），防止对方无响应时程序卡死；② 支持的城市以外输入，要给出友好提示。

产出：脚本 2，保存为脚本文件。

场景 3｜本机巡检工具雏形 ★（王牌项目版本 0 • 教学简化版）

本关三要素（编号 ｜ 时长 ｜ 完成判定）
3-14 • 场景 3（上）：本机巡检 v0——三个检查函数 ★ ｜ 休息日 4 小时 ｜ 工作日 20 分
□ 能输出磁盘 / 内存 / 负载三项
3-15 • 场景 3（下）：报告 + 日志 + 打包成项目 v0 ★ ｜ 休息日 3 小时 ｜ 工作日 15 分
□ 别人 clone 下来能跑出报告
实战价值：王牌项目第一版
工作日 15 分钟版：只做第 1 步 + 背 1 张卡（卡片见各阶段的工作日卡）。

📋 **背景工单**：领导要求每天对服务器做一次健康巡检并留档。你被指定做第一版：本机版巡检工具。⚠ **教学简化版说明：本场景的脚本只按 Ubuntu 的 df/free/uptime 预期输出做解析与验收；其他发行版（CentOS/Arch 等）的命令输出列数、字段名可能不同，跑出差异属于环境差异，不算你做错——验收只比较"解析思路与字段逻辑"，不比较固定数值。**这是你整条学习路线的王牌项目 server-health-check 的起点。（subprocess 调用模板已在示范题演示，按同样思路完成三个检查函数。）

🎯 **只给目标**（最终交付一个 health_check.py）：

目标	要求
A	用 subprocess 调用系统命令，实现三个检查函数：check_disk()（磁盘，调 df -h）、check_memory()（内存，调 free -m）、check_cpu()（负载，调 uptime）
B	每个函数解析命令输出，提取关键数值
C	设定阈值：磁盘使用率超 80%、内存使用率超 80% 时标记"告警"，否则"正常"
D	全部检查结果汇总成一份报告，**写入文件**留档（文件名带当天日期，日期用 time.strftime("%Y-%m-%d")（格式化时间）生成）

目标	要求
E	程序运行全过程用 `logging` 记日志（INFO 记开始/完成，写到文件）

⚠ 安全提示：`df`、`free`、`uptime` 是 Linux 命令（本场景标 🌐：须电脑环境完成；Windows 会报错）。只读命令，无破坏风险。

产出：server-health-check 版本 0，保存为脚本文件。

场景 4 | Nginx 日志分析器

本关三要素（编号 | 时长 | 完成判定）
3-16 • 场景 4：Nginx 日志分析器（样例日志） ★ | 休息日 3 小时 | 工作日 20 分
□ 统计出访问量前十的 IP
工作日 15 分钟版：只做第 1 步 + 背 1 张卡（卡片见各阶段的工作日卡）。

📋 **背景工单**：线上 Nginx 日志突然增长，领导要一份报告：谁在猛刷我们的网站、哪些页面 404 最多。**主线（必做）就用附录 B 的样例日志练**——它已包含全部解析要素。**Docker/Nginx 真实日志版是外部扩展**：本套教材不含 Docker 安装、镜像、日志挂载、端口与清理的前置，需要时自备环境，无 Docker、无网络不影响主线完成。

日志格式说明（题面给出）：Nginx 访问日志每行形如：

```
66.249.66.1 - - [18/Aug/2026:10:11:12 +0000] "GET /index.html HTTP/1.1" 200 612 "-" "Mozilla/5.0"
```

拆解：开头第一个字段是**访客 IP**；中间一对引号里是**请求行**（GET 方法 + URL + 协议）；请求行后面第一个数字是**状态码**（200 正常、404 找不到）。

🎯 **只给目标**（最终交付一个 `log_analyzer.py`）：

目标	要求
A	逐行读取日志文件（文件路径用 `argparse` 传入）
B	从每行提取访客 IP 与状态码（提示：请求行在一对引号里；状态码是请求行引号**之后、下一个引号之前**的裸字段）
C	统计访问量 Top 10 的 IP，降序输出
D	统计 404 最多的 5 个 URL
E	结果写入报告文件 `report.txt`

⚠ 安全提示：只读日志文件、只写 report.txt，无破坏操作。空行要 `strip` 后跳过，防止解析报错。

✎ 样例日志在附录 B（10 行，含重复 IP 和 404，直接复制保存成 access.log）。产出：脚本 3，保存为脚本文件。

场景 5 ☆ | paramiko 远程巡检雏形（选做 • 可跳过）

📋 **背景工单**：本机巡检跑通了，但真实服务器在远程。领导要求升级：从你的 Ubuntu（WSL 即可）SSH 到 Ubuntu 执行巡检。

⚠ **选做与跳过条件**：本场景为 ☆ 选做。需要你自己有一台可 SSH 的 Ubuntu（虚拟机即可）和一个可用账号；未安装 Paramiko、没有目标主机、或不具备 SSH 条件时，**直接跳过本场景，不影响主线学习**——它只是把场景 3 的本机巡检换成远程执行。

安装与连接说明（题面给出）：

项目	说明
安装	Ubuntu：`sudo apt install python3-paramiko`；Termux：`pkg install python -y` 后 `pip install paramiko`
前置准备	目标 Ubuntu 要先装并启动 SSH 服务：`sudo apt install openssh-server`、`sudo systemctl start ssh`；本机用 `ip addr` 查它的 IP
连接三要素	主机 IP、用户名、密码
paramiko 三步	建客户端 `paramiko.SSHClient()`（创建 SSH 客户端）→ `connect`(host, username, password)（建立连接）→ `exec_command`（远程执行命令，返回三个流）
首次连接	练习环境可跳过主机密钥验证（答案里有写法）；**生产环境必须验证**

🎯 **只给目标**：

目标	要求
A	用 `paramiko` 建立 SSH 连接：主机、用户名用 `input` 读入，密码用 `getpass.getpass` 隐藏输入；设置连接、认证和 banner 超时
B	远程执行 3~4 条巡检命令（如 `df -h`、`free -m`、`uptime`），分别读取 stdout、stderr 并检查退出码
C	输出汇总报告（每条命令前打印分隔标题）；认证失败、SSH 服务未启动、错误端口和远程命令失败都要友好结束，并用 `finally` 关闭连接

⚠ 安全提示：只连自己的虚拟机，**绝不连接生产服务器**；密码使用 `getpass.getpass` 读取且不回显；只连接自己的练习虚拟机，绝不使用真实生产密码。

产出：server-health-check 版本 2 雏形（选做：时间够就做）。

附录 A | 场景 1 测试文件生成

在 Ubuntu/Termux 终端里逐行执行（创建测试目录 + 10 个乱命名文件）：

```
$ mkdir -p ~/lab/renametest && cd ~/lab/renametest
$ touch "照片(1).jpg" IMG_20240601_001.jpg IMG_20240602_002.jpg IMG_20240603_003.jpg 旅游照片_西湖.jpg a.jpg b.jpg 截图2024.png IMG_20240604_004.jpg IMG_20240605_005.jpg
```

注意：含括号的中文文件名必须加引号（`"照片(1).jpg"`），否则 bash 会把 `()` 当元字符报 **`syntax error`**，整条命令都不会执行。共 9 个 .jpg（乱名）+ 1 个 .png（应被 `--ext jpg` 过滤掉）。备份：`mkdir ~/lab/renametest_backup && cp ~/lab/renametest/* ~/lab/renametest_backup/`

附录 B｜场景 4 样例日志（access.log）

把下面 10 行复制保存成 `access.log`（注意每行不能折行）：

```
66.249.66.1 - - [18/Aug/2026:10:11:12 +0000] "GET /index.html HTTP/1.1" 200 612 "-" "Mozilla/5.0"
203.0.113.9 - - [18/Aug/2026:10:11:15 +0000] "GET /style.css HTTP/1.1" 200 1024 "-" "Mozilla/5.0"
66.249.66.1 - - [18/Aug/2026:10:12:01 +0000] "GET /old-page HTTP/1.1" 404 180 "-" "Mozilla/5.0"
198.51.100.7 - - [18/Aug/2026:10:12:30 +0000] "GET /index.html HTTP/1.1" 200 612 "-" "curl/8.0"
66.249.66.1 - - [18/Aug/2026:10:13:02 +0000] "GET /favicon.ico HTTP/1.1" 404 150 "-" "Mozilla/5.0"
192.0.2.55 - - [18/Aug/2026:10:14:10 +0000] "GET /admin HTTP/1.1" 404 200 "-" "curl/8.0"
66.249.66.1 - - [18/Aug/2026:10:15:00 +0000] "GET /index.html HTTP/1.1" 200 612 "-" "Mozilla/5.0"
203.0.113.9 - - [18/Aug/2026:10:16:20 +0000] "GET /login HTTP/1.1" 200 890 "-" "Mozilla/5.0"
198.51.100.7 - - [18/Aug/2026:10:17:40 +0000] "GET /missing.jpg HTTP/1.1" 404 145 "-" "curl/8.0"
66.249.66.1 - - [18/Aug/2026:10:18:11 +0000] "GET /about.html HTTP/1.1" 200 700 "-" "Mozilla/5.0"
```

预期：66.249.66.1 访问 5 次（Top 1）；404 共 4 个：/old-page、/favicon.ico、/admin、/missing.jpg。

（本叠是题目，参考答案在另一叠；卡住 20 分钟以上再翻）